

Web服务器开发、文件上传

王红元 coderwhy

目录

content



1 Stream的读写操作

2 http模块web服务

3 request请求对象

4 response响应对象

5 axios node中使用

6 文件上传的细节分析

■ 什么是Stream（小溪、小河，在编程中通常翻译为流）呢？

- 我们的第一反应应该是流水，源源不断的流动；
- 程序中的流也是类似的含义，我们可以想象当我们从一个文件中读取数据时，文件的二进制（字节）数据会源源不断的被读取到我们程序中；
- 而这个一连串的字节，就是我们程序中的流；

■ 所以，我们可以这样理解流：

- 是连续字节的一种表现形式和抽象概念；
- 流应该是可读的，也是可写的；

■ 在之前学习文件的读写时，我们可以直接通过 readFile或者 writeFile方式读写文件，为什么还需要流呢？

- 直接读写文件的方式，虽然简单，但是无法控制一些细节的操作；
- 比如从什么位置开始读、读到什么位置、一次性读取多少个字节；
- 读到某个位置后，暂停读取，某个时刻恢复继续读取等等；
- 或者这个文件非常大，比如一个视频文件，一次性全部读取并不合适；



文件读写的Stream

- 事实上Node中很多对象是基于流实现的：

- [http](#)模块的Request和Response对象；

- 官方文档：另外所有的流都是EventEmitter的实例。

- 那么在Node中都有哪些流呢？

- Node.js中有四种基本流类型：

- [Writable](#)：可以向其写入数据的流（例如 [fs.createWriteStream\(\)](#)）。

- [Readable](#)：可以从中读取数据的流（例如 [fs.createReadStream\(\)](#)）。

- [Duplex](#)：同时为Readable和Writable（例如 [net.Socket](#)）。

- [Transform](#)：Duplex可以在写入和读取数据时修改或转换数据的流（例如[zlib.createDeflate\(\)](#)）。

- 这里我们通过fs的操作，讲解一下Writable、Readable，另外两个大家可以自行学习一下。

■ 之前我们读取一个文件的信息：

```
fs.readFile('./foo.txt', (err, data) => {  
  console.log(data);  
})
```

■ 这种方式是一次性将一个文件中所有的内容都读取到程序（内存）中，但是这种读取方式就会出现我们之前提到的很多问题：

- ❑ 文件过大、读取的位置、结束的位置、一次读取的大小；

■ 这个时候，我们可以使用 `createReadStream`，我们来看几个参数，更多参数可以参考官网：

- ❑ `start`：文件读取开始的位置；

- ❑ `end`：文件读取结束的位置；

- ❑ `highWaterMark`：一次性读取字节的长度，默认是64kb；

Readable的使用

■ 创建文件的Readable

```
const read = fs.createReadStream("./foo.txt", {  
  start: 3,  
  end: 8,  
  highWaterMark: 4  
});
```

■ 我们如何获取到数据呢?

- 可以通过监听data事件，获取读取到的数据；

```
read.on("data", (data) => {  
  console.log(data);  
});
```

■ 也可以做一些其他的操作：监听其他事件、暂停或者恢复

```
read.on('open', (fd) => {  
  console.log("文件被打开");  
})  
  
read.on('end', () => {  
  console.log("文件读取结束");  
})  
  
read.on('close', () => {  
  console.log("文件被关闭");  
})
```

```
read.pause();  
setTimeout(() => {  
  read.resume();  
}, 2000);
```



Writable

- 之前我们写入一个文件的方式是这样的：

```
fs.writeFile('./foo.txt', "内容", (err) => {  
  ..  
});
```

- 这种方式相当于一次性将所有的内容写入到文件中，但是这种方式也有很多问题：

- 比如我们希望一点点写入内容，精确每次写入的位置等；

- 这个时候，我们可以使用 `createWriteStream`，我们来看几个参数，更多参数可以参考官网：

- flags: 默认是w，如果我们希望是追加写入，可以使用 a或者 a+；
- start: 写入的位置；

Writable的使用

■ 我们进行一次简单的写入

```
const writer = fs.createWriteStream("./foo.txt", {  
  flags: "a+",  
  start: 8  
});  
  
writer.write("你好啊", err => {  
  console.log("写入成功");  
});
```

■ 你可以监听open事件:

```
writer.on("open", () => {  
  console.log("文件打开");  
})
```

close的监听

■ 我们会发现，我们并不能监听到 close 事件：

- 这是因为写入流在打开后是不会自动关闭的；
- 我们必须手动关闭，来告诉Node已经写入结束了；
- 并且会发出一个 finish 事件的；

■ 另外一个非常常用的方法是 end：end方法相当于做了两步操作： write传入的数据和调用close方法；

```
writer.close();

writer.on("finish", () => {
  console.log("文件写入结束");
})

writer.on("close", () => {
  console.log("文件关闭");
})
```

```
writer.end("Hello World");
```

pipe方法

- 正常情况下，我们可以将读取到的 输入流，手动的放到 输出流中进行写入：

```
const reader = fs.createReadStream('./foo.txt');
const writer = fs.createWriteStream('./bar.txt');

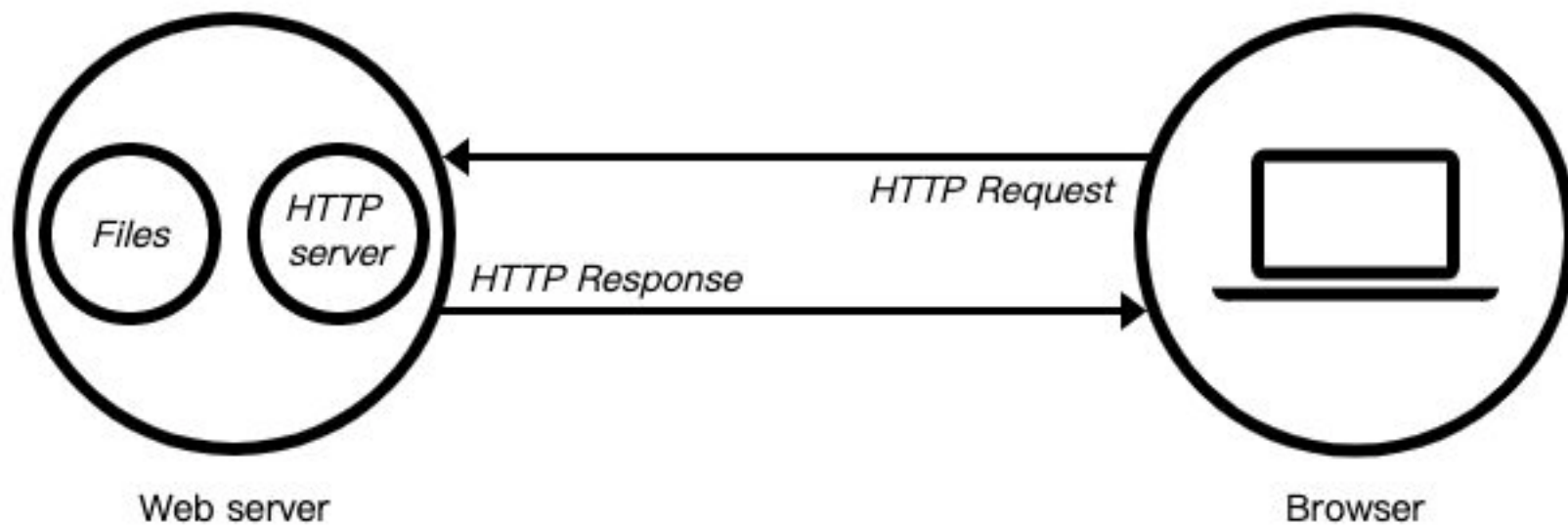
reader.on("data", (data) => {
  console.log(data);
  writer.write(data, (err) => {
    console.log(err);
  });
});
```

- 我们也可以通过pipe来完成这样的操作：

```
reader.pipe(writer);
```

■ 什么是Web服务器？

- 当应用程序（客户端）需要某一个资源时，可以向一台服务器，通过Http请求获取到这个资源；
- 提供资源的这个服务器，就是一个Web服务器；



- 目前有很多开源的Web服务器：Nginx、Apache（静态）、Apache Tomcat（静态、动态）、Node.js

- 在Node中，提供web服务器的资源返回给浏览器，主要是通过http模块。
- 我们先简单对它做一个使用：

```
const http = require('http');

const HTTP_PORT = 8000;

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(8000, () => {
  console.log(`🚀 服务器在${HTTP_PORT}启动~`)
})
```

创建服务器

■ 创建服务器对象，我们是通过 createServer 来完成的

- http.createServer会返回服务器的对象；
- 底层其实使用直接 new Server 对象。

```
function createServer(opts, requestListener) {  
  return new Server(opts, requestListener);  
}
```

■ 那么，当然，我们也可以自己来创建这个对象：

```
const server2 = new http.Server((req, res) => {  
  res.end("Hello Server2");  
});  
server2.listen(9000, () => {  
  console.log("服务器启动成功~");  
})
```

■ 上面我们已经看到，创建Server时会传入一个回调函数，这个回调函数在被调用时会传入两个参数：

- req: request请求对象，包含请求相关的信息；
- res: response响应对象，包含我们要发送给客户端的信息；

监听主机和端口号

- **Server**通过**listen**方法来开启服务器，并且在某一个主机和端口上监听网络请求：
 - 也就是当我们通过 **ip:port**的方式发送到我们监听的**Web服务器**上时；
 - 我们就可以**对其进行相关的处理**；
- **listen**函数有三个参数：
 - **端口port**: 可以不传, 系统会默认分配端, 后续项目中我们会写入到环境变量中；
 - **主机host**: 通常可以传入localhost、ip地址127.0.0.1、或者ip地址0.0.0.0，默认是0.0.0.0；
 - **localhost**: 本质上是一个域名，通常情况下会被解析成127.0.0.1；
 - **127.0.0.1**: 回环地址（Loop Back Address），表达的意思其实是我们主机自己发出去的包，直接被自己接收；
 - ✓ 正常的数据库包经常 应用层 - 传输层 - 网络层 - 数据链路层 - 物理层；
 - ✓ 而回环地址，是在网络层直接就被获取到了，是不会经常数据链路层和物理层的；
 - ✓ 比如我们监听 127.0.0.1时，在同一个网段下的主机中，通过ip地址是不能访问的；
 - **0.0.0.0**:
 - ✓ 监听IPV4上所有的地址，再根据端口找到不同的应用程序；
 - ✓ 比如我们监听 0.0.0.0时，在同一个网段下的主机中，通过ip地址是可以访问的；
 - **回调函数**: 服务器启动成功时的回调函数；

request对象

■ 在向服务器发送请求时，我们会携带很多信息，比如：

- 本次请求的URL，服务器需要根据不同的URL进行不同的处理；
- 本次请求的请求方式，比如GET、POST请求传入的参数和的方式是不同的；
- 本次请求的headers中也会携带一些信息，比如客户端信息、接受数据的格式、支持的编码格式等；
- 等等...

■ 这些信息，Node会帮助我们封装到一个request的对象中，我们可以直接来处理这个request对象：

```
const server = http.createServer((req, res) => {  
  // request对象  
  console.log(req.url);  
  console.log(req.method);  
  console.log(req.headers);  
  
  res.end("Hello World");  
});
```

URL的处理

■ 客户端在发送请求时，会请求不同的数据，那么会传入不同的请求地址：

- 比如 `http://localhost:8000/login;`
- 比如 `http://localhost:8000/products;`

■ 服务器端需要根据不同的请求地址，作出不同的响应：

```
const server = http.createServer((req, res) => {  
  const url = req.url;  
  console.log(url);  
  
  if (url === '/login') {  
    res.end("welcome Back~");  
  } else if (url === '/products') {  
    res.end("products");  
  } else {  
    res.end("error message");  
  }  
});
```

URL的解析

■ 那么如果用户发送的地址中还携带一些额外的参数呢？

- `http://localhost:8000/login?name=why&password=123;`
- 这个时候，url的值是 `/login?name=why&password=123;`

■ 我们如何对它进行解析呢？使用内置模块url：

```
const parseInfo = url.parse(req.url);  
console.log(parseInfo);
```

■ 但是 query 信息如何可以获取呢？

```
const { pathname, query } = url.parse(req.url);  
  
const queryObj = qs.parse(query);  
console.log(queryObj.name);  
console.log(queryObj.password);
```

■ 在Restful规范（设计风格）中，我们对于数据的增删改查应该通过不同的请求方式：

- GET：查询数据；
- POST：新建数据；
- PATCH：更新数据；
- DELETE：删除数据；

■ 所以，我们可以通过判断不同的请求方式进行不同的处理。

- 比如创建一个用户：
- 请求接口为 /users；
- 请求方式为 POST请求；
- 携带数据 username和密码；

■ 在我们程序中如何进行判断以及获取对应的数据呢？

- 这里我们需要判断接口是 /users，并且请求方式是POST方法去获取传入的数据；
- 获取这种body携带的数据，我们需要通过监听req的 data事件来获取；

```
req.setEncoding('utf-8');

req.on('data', (data) => {
  const {username, password} = JSON.parse(data);
  console.log(username, password);
});

req.on("end", () => {
  console.log("传输结束");
})

res.end("create user success");
```

■ 将JSON字符串格式转成对象类型，通过JSON.parse方法即可。

HTTP Request Header (一)

- 在request对象的header中也包含很多有用的信息，客户端会默认传递过来一些信息：

▼ Request Headers

Accept: */*

Accept-Encoding: gzip, deflate

Accept-Language: zh-CN,zh;q=0.9

Access-Control-Request-Headers: token

Access-Control-Request-Method: POST

Connection: keep-alive

Host: 192.168.0.110:1888

Origin: http://127.0.0.1:5500

Referer: http://127.0.0.1:5500/

Sec-Fetch-Mode: cors

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/102.0.0.0 Safari/537.36

- **content-type**是这次请求携带的数据的类型：

- ❑ **application/x-www-form-urlencoded**：表示数据被编码成以 '&' 分隔的键 - 值对，同时以 '=' 分隔键和值
- ❑ **application/json**：表示是一个json类型；
- ❑ **text/plain**：表示是文本类型；
- ❑ **application/xml**：表示是xml类型；
- ❑ **multipart/form-data**：表示是上传文件；



HTTP Request Header (二)



■ **content-length**: 文件的大小长度

■ **keep-alive**:

- http是基于TCP协议的，但是通常在进行一次请求和响应结束后会立刻中断；

- 在http1.0中，如果想要继续保持连接：

 - ✓ 浏览器需要在请求头中添加 `connection: keep-alive`；

 - ✓ 服务器需要在响应头中添加 `connection:keep-alive`；

 - ✓ 当客户端再次放请求时，就会使用同一个连接，直接一方中断连接；

- 在http1.1中，所有连接默认是 `connection: keep-alive`的；

 - ✓ 不同的Web服务器会有不同的保持 `keep-alive`的时间；

 - ✓ Node中默认是5s中；

■ **accept-encoding**: 告知服务器，客户端支持的文件压缩格式，比如js文件可以使用gzip编码，对应 .gz文件；

■ **accept**: 告知服务器，客户端可接受文件的格式类型；

■ **user-agent**: 客户端相关的信息；

返回响应结果

- 如果我们希望给客户端响应的结果数据，可以通过两种方式：

- **Write方法**：这种方式是直接写出数据，但是并没有关闭流；
- **end方法**：这种方式是写出最后的数据，并且写出后会关闭流；

```
// 响应数据的方式有两个：  
res.write("Hello World");  
res.write("Hello Response");  
res.end("message end");
```

- 如果我们没有调用 **end**和**close**，客户端将会一直等待结果：

- 所以客户端在发送网络请求时，都会设置超时时间。

返回状态码

■ **Http状态码 (Http Status Code)** 是用来表示Http响应状态的数字代码：

□ Http状态码非常多，可以根据不同的情况，给客户端返回不同的状态码；

□ MDN响应码解析地址：<https://developer.mozilla.org/zh-CN/docs/web/http/status>

常见HTTP状态码	状态描述	信息说明
200	OK	客户端请求成功
201	Created	POST请求，创建新的资源
301	Moved Permanently	请求资源的URL已经修改，响应中会给出新的URL
400	Bad Request	客户端的错误，服务器无法或者不进行处理
401	Unauthorized	未授权的错误，必须携带请求的身份信息
403	Forbidden	客户端没有权限访问，被拒接
404	Not Found	服务器找不到请求的资源。
500	Internal Server Error	服务器遇到了不知道如何处理的情况。
503	Service Unavailable	服务器不可用，可能处理维护或者重载状态，暂时无法访问

```
res.statusCode = 400;  
res.writeHead(200);
```

响应头文件

■ 返回头部信息，主要有两种方式：

- `res.setHeader`：一次写入一个头部信息；
- `res.writeHead`：同时写入header和status；

```
res.setHeader("Content-Type", "application/json;charset=utf8");  
  
res.writeHead(200, {  
  "Content-Type": "application/json;charset=utf8"  
})
```

■ Header设置 Content-Type有什么作用呢？

- 默认客户端接收到的是字符串，客户端会按照自己默认的方式进行处理；

← → ↻ ⓘ localhost:8000

<h2>Hello World</h2>

← → ↻ ⓘ localhost:8000

Hello World

http请求

■ axios库可以在浏览器中使用，也可以在Node中使用：

- 在浏览器中，axios使用的是封装xhr；
- 在Node中，使用的是http内置模块；

```
http.get("http://localhost:8000", (res) => {  
  res.on('data', data => {  
    console.log(data.toString());  
    console.log(JSON.parse(data.toString()));  
  })  
});
```

```
const req = http.request({  
  method: 'POST',  
  hostname: 'localhost',  
  port: 8000  
}, (res) => {  
  res.on('data', data => {  
    console.log(data.toString());  
    console.log(JSON.parse(data.toString()));  
  })  
})  
  
req.end();
```

文件上传 - 错误示范

- 如果是一个很大的文件需要上传到服务器端，服务器端进行保存应该如何操作呢？

```
const fileWriter = fs.createWriteStream('./foo.png');
req.pipe(fileWriter);

const fileSize = req.headers['content-length'];
let curSize = 0;
console.log(fileSize);

req.on("data", (data) => {
  curSize += data.length;
  console.log(curSize);
  res.write(`文件上传进度: ${curSize/fileSize * 100}%\n`);
});

req.on('end', () => {
  res.end("文件上传完成~");
})
```

文件上传 – 正确做法 – 代码片段一

```
// 图片文件必须设置为二进制的
req.setEncoding('binary');

// 获取content-type中的boundary的值
var boundary = req.headers['content-type'].split(';')[1].replace('boundary=', '');

// 记录当前数据的信息
const fileSize = req.headers['content-length'];
let curSize = 0;
let body = '';

// 监听当前的数据
req.on("data", (data) => {
  curSize += data.length;
  res.write(`文件上传进度: ${curSize/fileSize * 100}%\n`);
  body += data;
});
```

文件上传 – 正确做法 – 代码片段二

```
// 数据结构
req.on('end', () => {
  // 切割数据
  const payload = qs.parse(body, "\r\n", ":");
  // 获取最后的类型(image/png)
  const fileType = payload["Content-Type"].substring(1);
  // 获取要截取的长度
  const fileTypePosition = body.indexOf(fileType) + fileType.length;
  let binaryData = body.substring(fileTypePosition);
  binaryData = binaryData.replace(/^\s\s*/, '');

  // binaryData = binaryData.replaceAll('\r\n', '');
  const finalData = binaryData.substring(0, binaryData.indexOf('--'+boundary+'--'));

  fs.writeFile('./boo.png', finalData, 'binary', (err) => {
    console.log(err);
    res.end("文件上传完成~");
  })
})
```